

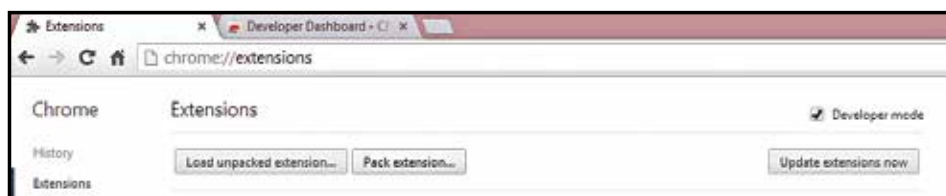
5. A Google account (preferably a dedicated account for development purposes);

Let's start!

1. Launch Sublime Text 2
2. Go to Tools > Command Palette... [Ctrl]+[Shift]+[P]
3. Search "install"
4. Click "Package Control" and install Package (If you don't have Package Control, you can get it from: <http://dgit.in/1f4RWBN>)
5. Search "Chrome" in the Package Control dialogue box
6. Click "Chrome Apps and Extensions"
7. Notice a new tab on the right side of Help called "Chrome"
8. Go to Chrome > New App... > Hello World!
9. Three files are created. Save each of them in the same directory to enable colour-coding. (These three files are typical to any packaged app on Chrome.



This is how the Developer Dashboard looks like.



The Extensions page in Chrome.

For our purposes, we'll need only two files: manifest.json and background.js

manifest.json:

```
{
  "name" : "Digit",
  "description" : "To use, type 'digit' [space] &
  a search term into the address bar.",
  "version" : "1.0",
  "background":
  {
    "scripts" : ["background.js"]
  },
  "omnibox" : { "keyword" : "digit" },
  "manifest_version" : 2
}
```

The manifest describes meta-information about your application such as name, one-line description, version number (to avoid duplicity between updates) and additional information

on how to launch your app and privileges associated with it. Manifest version 1 has been deprecated and is no longer available for use. You can find out more about the manifest file at <http://dgit.in/1fhVdni>

background.js:

```
//This event is triggered when the user changes
the input in the omnibox.
chrome.omnibox.onInputChanged
  addListener(function(search, suggest)
{
  console.log('Input Changed: ' + search);
  suggest
  ([
    {content: text + " change1", description:
    "first suggestion"},
    {content: text + " change2", description:
    "second suggestion"}
  ]);
});
//This event is triggered when
the user accepts the input in
the Omnibox.
chrome.omnibox.onInputEntered.addListener(
function(text)
{
  console.log('Input Entered:
  ' + text);
  alert('You searched for "' +
  text + '"');
});
```

The Omnibox module is utilised in this code to run as a background script, since the application doesn't have its own dedicated UI. The two functions called in the code are essentially event listeners that are triggered

when the input is changed or entered into the Omnibox. Both the functions are defined using anonymous function definitions. "console.log()" function is used to show output on the development console and not to the user. "suggest()" function accepts an array and can be programmed to receive its data from a large website equipped with search engine capabilities. It accepts data with two parameters, that is 'text' – the corresponding replacement text for each suggestion in the Omnibox that changes the input in the address bar when any suggestion is hovered over and 'description' – the suggestion option that appear in the Omnibox, which could even be the title of a webpage. When the user accepts the search text by hitting [enter]. The alert() function displays a pop-up confirming that the Omnibox is accepting the input from the user.

10. Once you've completed saving the two files in a folder, open the Chrome browser
11. Go to Options -> Tools -> Extensions or type "chrome://extensions/" in the address bar
12. Make sure Developer Mode is checked